コンパイラ (11)

2026/11/10

静的単一代入を用いた最適化

データフロー解析のもうひとつの手法: 静的単一代入 (Static Single Assignment:SSA)

各変数の定義がプログラム中で唯一

変数の値は変わらないとする。二度目の代入が発生すると変数名を変更

　値が同じの共通部分式が判別できる=式として一致

変換例：例 6.20、6.21

定義ごとに変数に添字をつける。合流する場合は、ϕ 関数で表現

いかにして、SSA 形式を得るか？

(1) ϕ 関数の挿入

(2) 変数の添字の決定

SSA 形式において最適化

(3) SSA から逆変換する。(ϕ 関数の消去)

近年のコンパイラでよく用いられる手法

例

```
scanf("%d",&x);
a=1;
b=2;
c=a+b;
if(x>0) a=3;
d=b;
c=a+b;
```

⇒

```
scanf("%d",&x0);
a0=1;
b0=2;
c0=a0+b0;
if(x>0) a1=3;
a2=φ(a0,a1);
d0=b0;
c1=a2+b0;
```

```
t=1
n=10
l1:  t=t*n
n=n-1
b=n>1
if b goto l1
f=t
```

⇒

```
    t1=1
    n1=10
l1: t2=φ(t1,t3)
    n2=φ(n1,n3)
    t3=t2*n2
    n3=n2-1
    b1=n3>1
    if b1 goto l1
    f1=t3
```

φ 関数の挿入

フローグラフ (V, E, v_{init})：$v \in V$ は，v_{init} から到達可能．

定義 7.1(定義合流条件)　変数 x に対する φ 関数が頂点 v に挿入 $\Leftrightarrow$ 以下に示す条件 (1)~(4) を満たすコントロールグラフ上の路 $p : v_0 \to v_1 \to \cdots \to v_k$, $q : u_0 \to u_1 \to \cdots \to u_\ell$ が存在．

(1) 路の長さが 1 以上である．$(k > 0, \ell > 0)$;

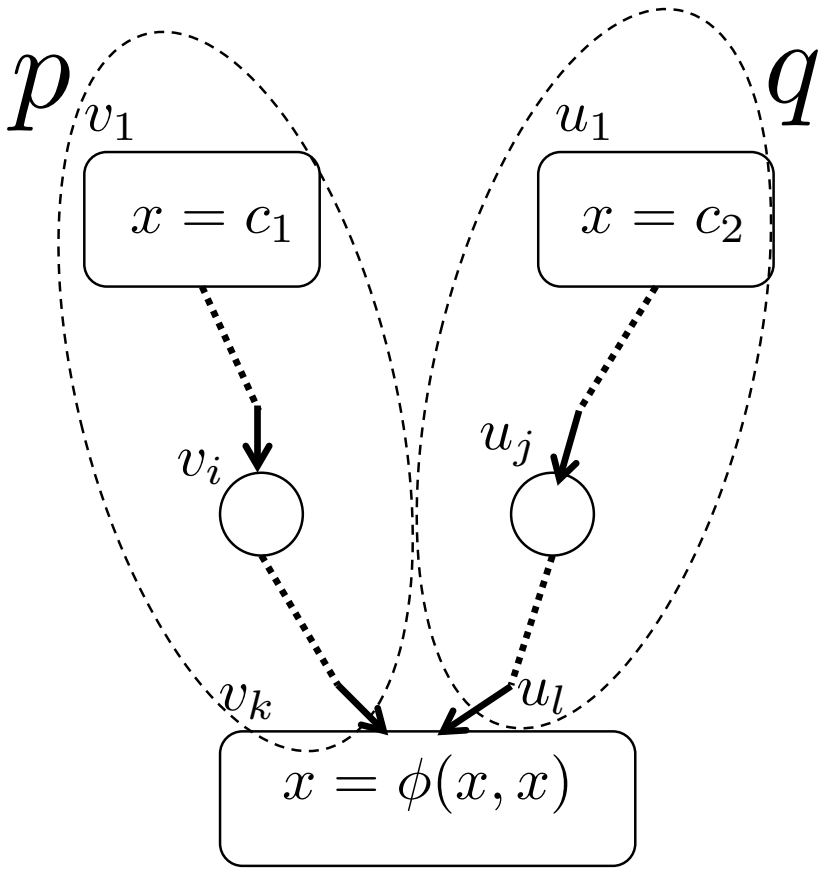
(2) v_1 と u_1 の両方に x の定義が存在する．;

(3) 路の終点 v が一致している．$(v_k = v_\ell = v)$;

(4) 路の途中で交わることがない．
$(\{v_1, \cdots, v_{k-1}\} \cap \{u_1, \cdots, v_{\ell-1}\} = \varnothing)$

パス p 上の x の定義$=x_1$,q 上の x の定義$=x_2$ 定義合流条件の v に、$x_3 = \phi(x_1, x_2)$

定義合流条件



支配関係

支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。

定義 7.2(支配関係)　フローグラフを (V, E, v_{init}) とする．頂点 v, u に対して，v_{init} から u へのすべての路において v が出現するとき，v は u の支配頂点 **(dominator)** であると呼び，$u \underline{\ll} v$ と書く．$u \neq v$ かつ $u \underline{\ll} v$ であるとき，$u \ll v$ と書く．u の支配頂点の集合 $\mathrm{dom}(u) = \{v \mid u \underline{\ll} v\}$ で定義する．

$u \underline{\gg} v$:u が v を支配: 支配関係の表現=支配木

支配関係

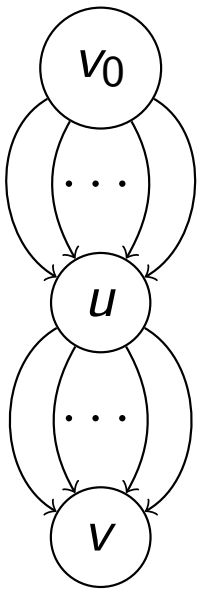
支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。
どの頂点 v についても $v_{init} \in \mathrm{dom}(v)$ かつ $v \in \mathrm{dom}(v)$ である．$\underline{\ll}$ は半順序である．

- (反射律) $v \underline{\ll} v$
- (反対称律) $v \underline{\ll} u$ かつ $u \underline{\ll} v$ ならば $u = v$
- (推移律) $v \underline{\ll} u$ かつ $u \underline{\ll} w$ ならば $v \underline{\ll} w$

$u \neq v_{init}$ のとき，$\mathrm{dom}(u) \backslash \{u\}$ には $\ll$ の意味で最小の頂点が一つ存在する．

支配関係

支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。



$u \underline{\gg} v$

$u \underline{\gg} u$

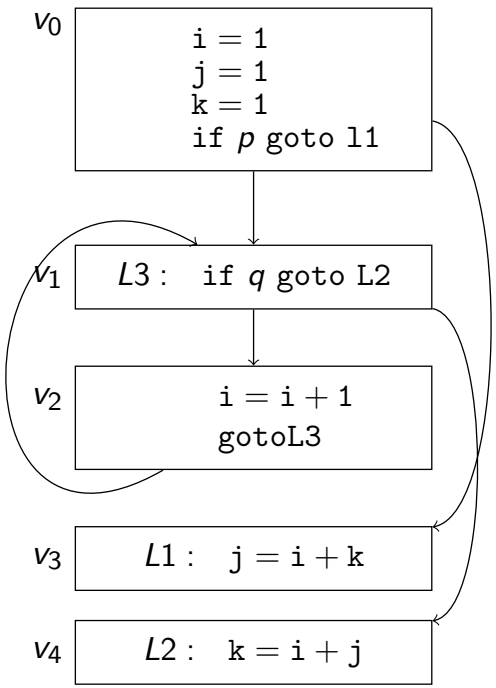
$u \underline{\gg} v, v \underline{\gg} w$ ならば $u \underline{\gg} w$

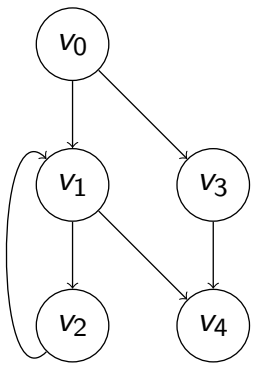
$u \neq v$ のとき $u \gg v$

v 以外で最も下にある支配頂点 u: $idom(v)$

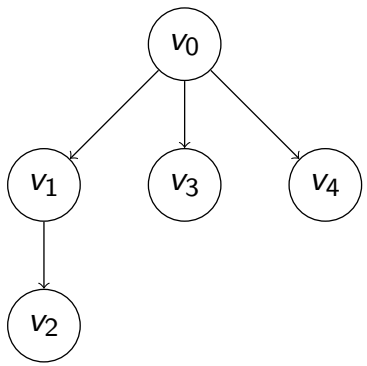
$(idom(v), v)$ を辺とするグラフ＝支配木

フローグラフ → 支配木





フローグラフ



支配木

静的単一代入を用いた最適化

データフロー解析のもうひとつの手法:
静的単一代入 (Static Single Assignment:SSA)
各変数の定義がプログラム中で唯一
二度目の代入が発生すると変数名を変更
　値が同じの共通部分式が判別できる=式として一致
定義ごとに変数に添字をつける。合流する場合は、φ 関数で表現
いかにして、SSA 形式を得るか？

(1) φ 関数の挿入
(2) 変数の添字の決定

　　SSA 形式において最適化

(3) SSA から逆変換する。(φ 関数の消去)

近年のコンパイラでよく用いられる手法

例

```
scanf("%d",&x);
a=1;
b=2;
c=a+b;
if(x>0) a=3;
d=b;
c=a+b;
```

⇒

```
scanf("%d",&x0);
a0=1;
b0=2;
c0=a0+b0;
if(x>0) a1=3;
a2=φ(a0,a1);
d0=b0;
c1=a2+b0;
```

```
t=1
n=10
l1:  t=t*n
n=n-1
b=n>1
if b goto l1
f=t
```

⇒

```
    t1=1
    n1=10
l1: t2=φ(t1,t3)
    n2=φ(n1,n3)
    t3=t2*n2
    n3=n2-1
    b1=n3>1
    if b1 goto l1
    f1=t3
```

φ 関数の挿入

$\mathtt{x}=\varphi(\mathtt{x},\cdots,\mathtt{x})$ の挿入：

- 変数 $\mathtt{x}$ に対して定義 *7.1* の条件を満たす v_0，u_0，v がフローグラフに存在し，頂点 v の基本ブロックに φ 関数が含まれていないならば，v に $\mathtt{x}=\varphi(\mathtt{x},\ldots,\mathtt{x})$ を挿入する．
- 定義合流条件を満たす v_0，u_0，v がなくなるまで，上記を繰り返す．

支配辺境

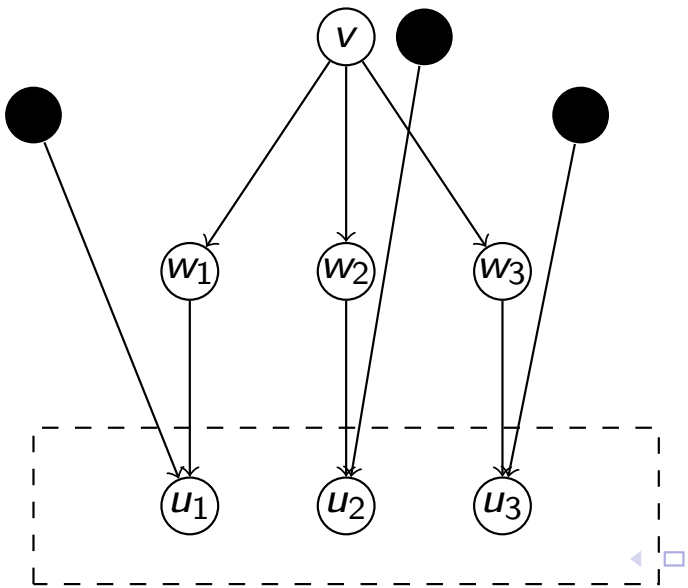
フローグラフ $G = (V, E)$

定義

v の支配辺境 *(Dominance Frontier)*

$$DF(v) = \{u \in V | \exists w.w \in pred(u), v \geq w, v \not\gg u\}$$

強く支配していれば ϕ 関数を挿入する必要なし



支配辺境の計算

フローグラフ $G = (V, E, v_{init})$

補題

$$DF(v) = DF_{local}(v) \cup \bigcup_{w \in Child(v)} DF^{v}_{up}(w)$$

ここで、

$$DF_{local}(v) = \{u | (v, u) \in E, \mathrm{idom}(u) \neq v\}$$

v の次の頂点で他からのフローが存在する

$$DF^{v}_{up}(w) = \{u \in DF(w) | \mathrm{idom}(u) \neq v\}$$

w は支配木において v の子で u から見て、
v は w よりも上 (根に近い) にある。

支配木の葉から DF を計算していく (p129：例)

定義合流条件の計算

定義合流条件'(S) 二つのパス $p(= v_1, \cdots, v_k)$, $q(= u_1, \cdots, u_l)$

(1) $k \geq 2$, $l \geq 2$
(2) $v_1, u_1 \in S$
(3) $v_k = u_l = v$
(4) $\{v_1, \cdots, v_{k-1}\} \cap \{u_1, \cdots, u_{l-1}\} = \emptyset$

$J(S) = \{v |$ 上の条件を満たす $v\}$

変数 x が定義される場所の集合を S とすると $J(S)$ は、x に対して最初に ϕ 関数 $x = \phi(x, x)$ が挿入される場所

ϕ 関数によって新たに定義が挿入されるため、さらに挿入が必要: 次の漸化式で不動点 $J^+(S_0)$ を計算

$$\begin{aligned} S_{i+1} &= J(S_0 \cup S_i) \\ J^{n+1}(S_0) &= J(S_0 \cup J^n(S_0)) \\ J^+(S_0) &= \lim_{n \leftarrow \infty} J^n(S_0) \end{aligned}$$

支配辺境と合流条件

定理

$DF(S) = \bigcup_{v \in S} DF(v)$ **のとき、**

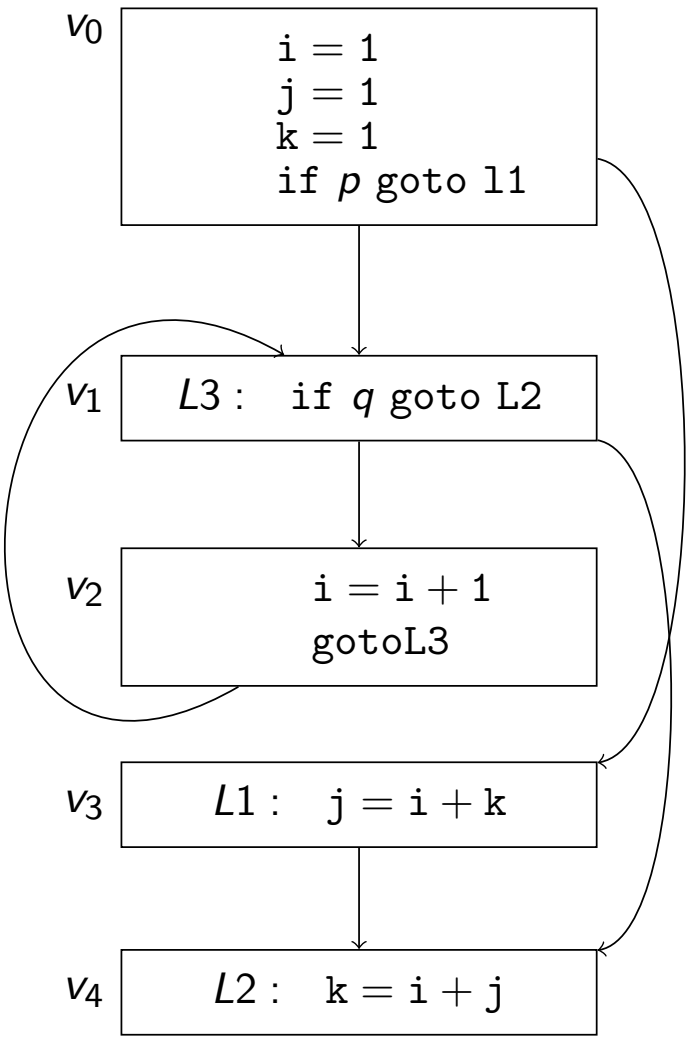
$$F_{i+1} = DF(S_0 \cup F_i) \quad (i \geq 0, F_0 = \emptyset)$$

とするとき， $DF^+(S_0) = \lim_{n \leftarrow \infty} F_n$ **とする．このとき，**

$$DF^+(S_0) = J^+(S_0)$$

合流条件は支配辺境から計算可能

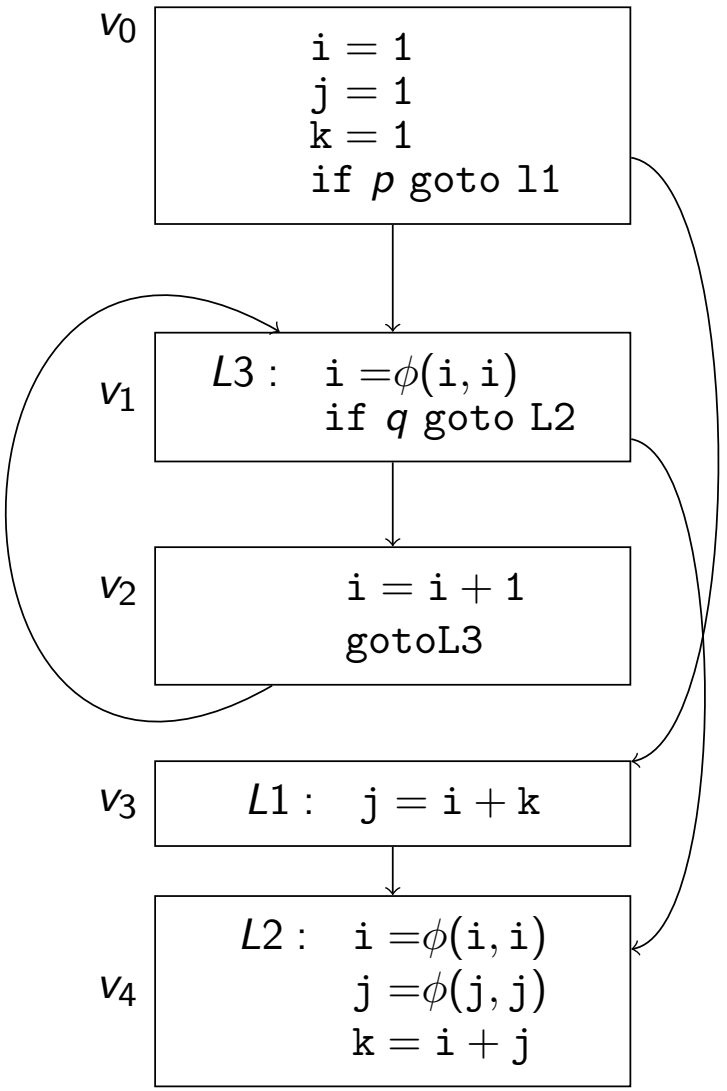
例 6.29



i の定義：$S_0 = \{v_0, v_2\}$
j の定義：$S_0' = \{v_0, v_3\}$
k の定義：$S_0'' = \{v_0, v_4\}$

$$DF^+(S_0) = \{v_1, v_4\}$$
$$DF^+(S_0') = \{v_4\}$$
$$DF^+(S_0'') = \emptyset$$

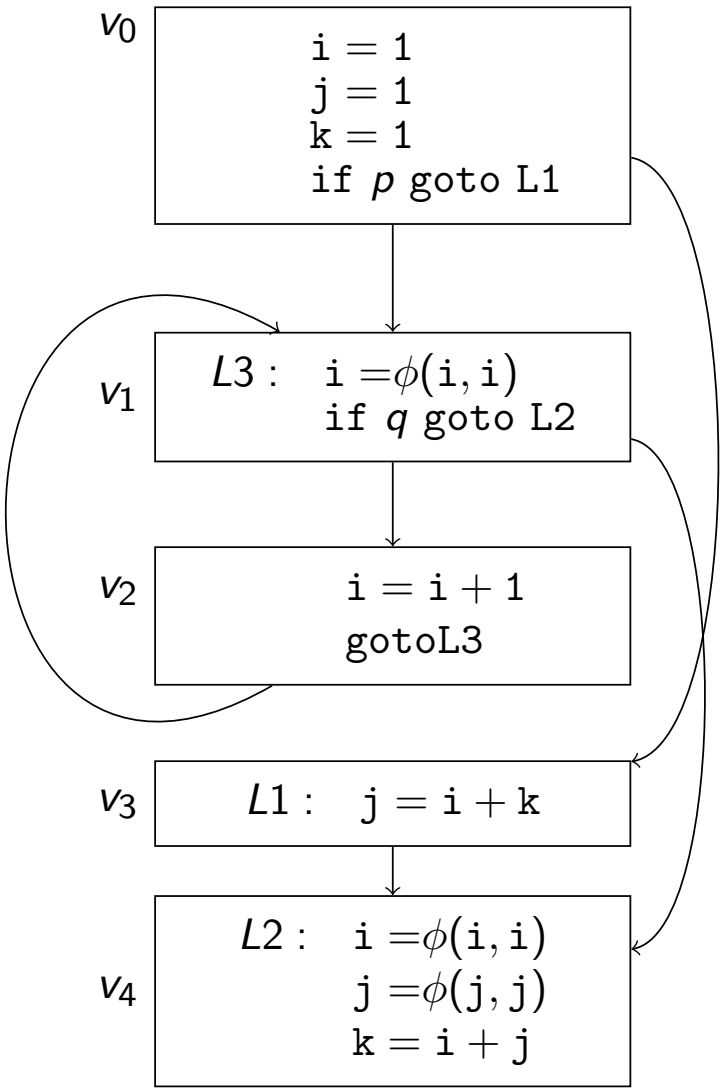
例 6.29

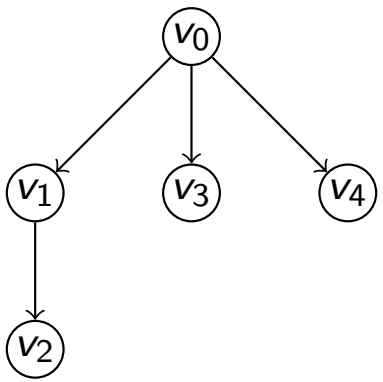


i の定義：$S_0 = \{v_0, v_2\}$
j の定義：$S_0' = \{v_0, v_3\}$
k の定義：$S_0'' = \{v_0, v_4\}$

$$DF^+(S_0) = \{v_1, v_4\}$$
$$DF^+(S_0') = \{v_4\}$$
$$DF^+(S_0'') = \emptyset$$

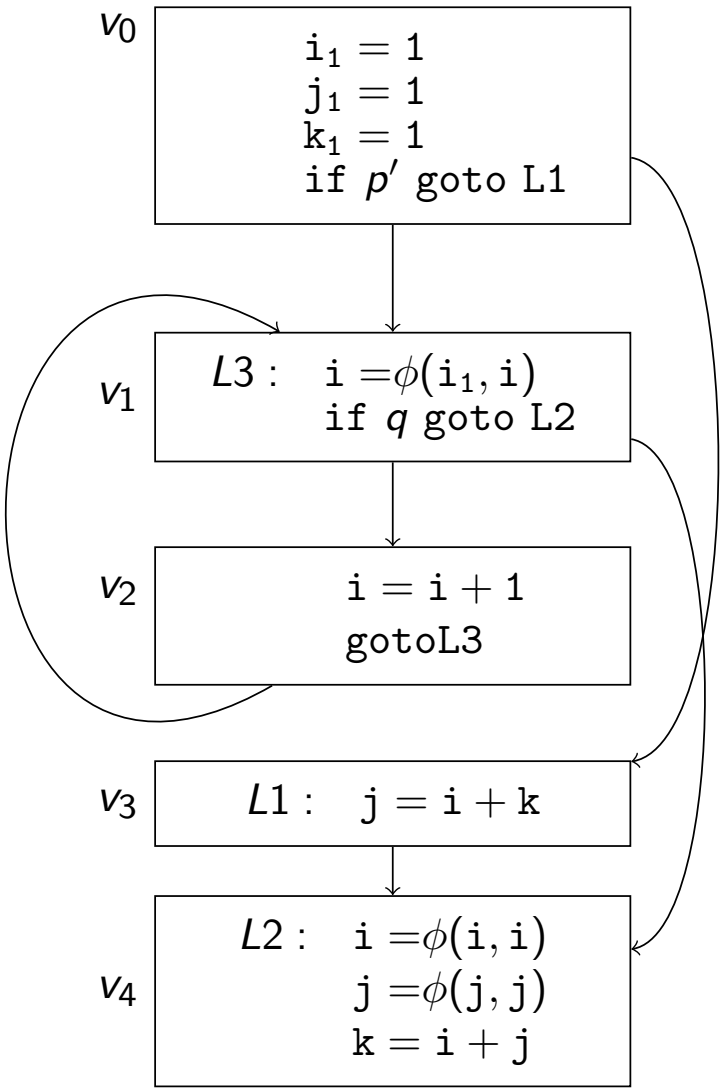
SSA 形式への変換

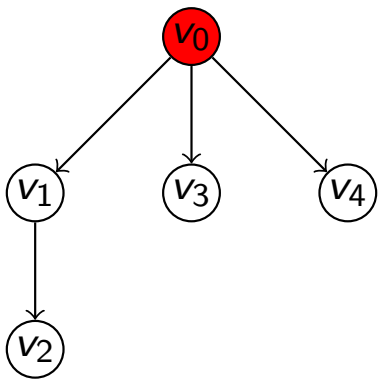




$$stack(i) = [] \quad c(i) = 1$$
$$stack(j) = [] \quad c(j) = 1$$
$$stack(k) = [] \quad c(k) = 1$$

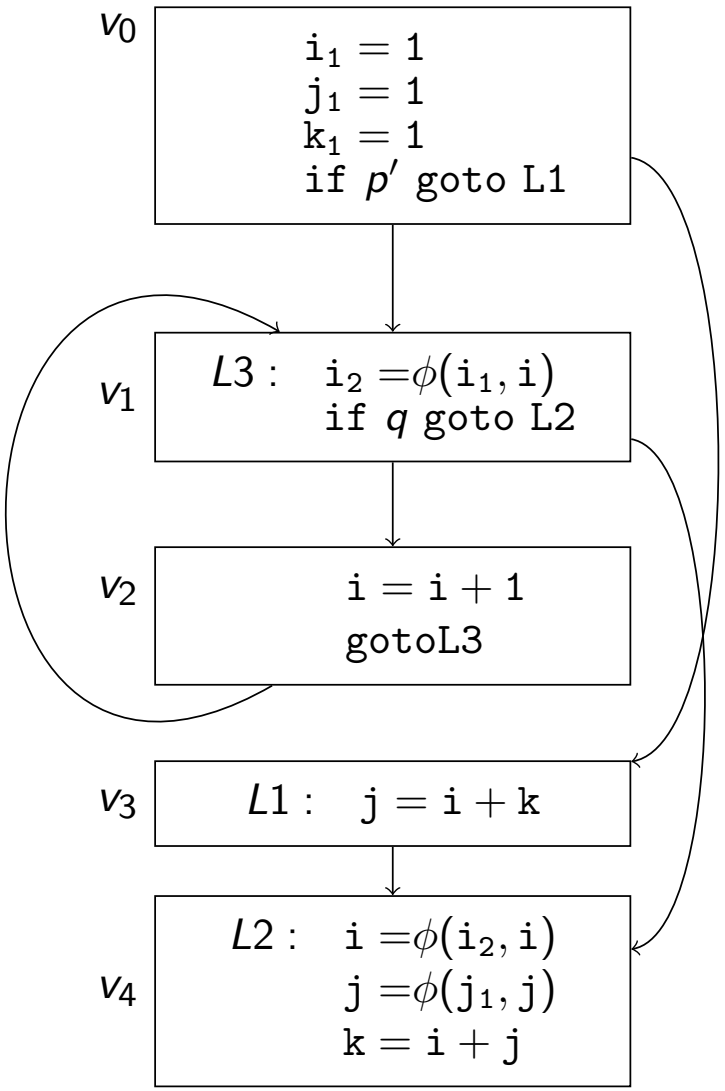
SSA 形式への変換

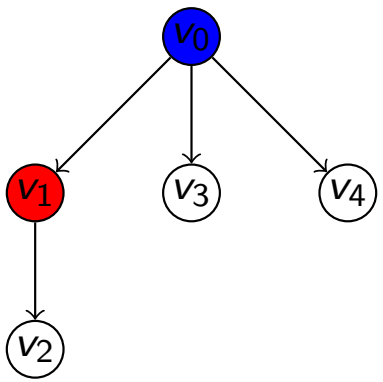




$$stack(i) = [1] \quad c(i) = 2$$
$$stack(j) = [1] \quad c(j) = 2$$
$$stack(k) = [1] \quad c(k) = 2$$

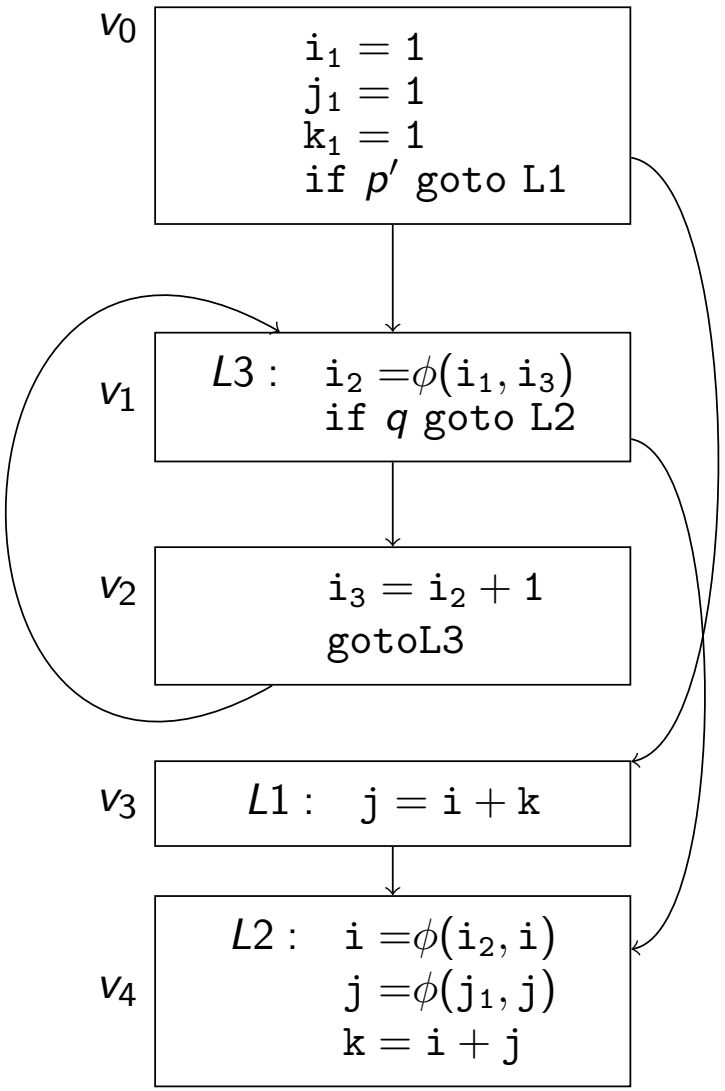
SSA 形式への変換

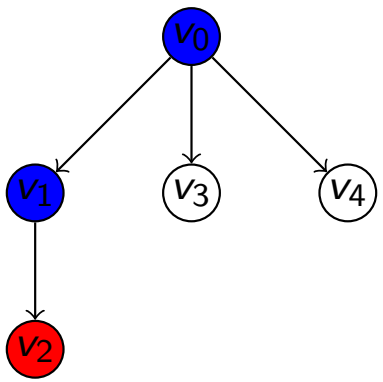




$stack(i) = [1, 2]$ $c(i) = 3$
$stack(j) = [1]$ $c(j) = 2$
$stack(k) = [1]$ $c(k) = 2$

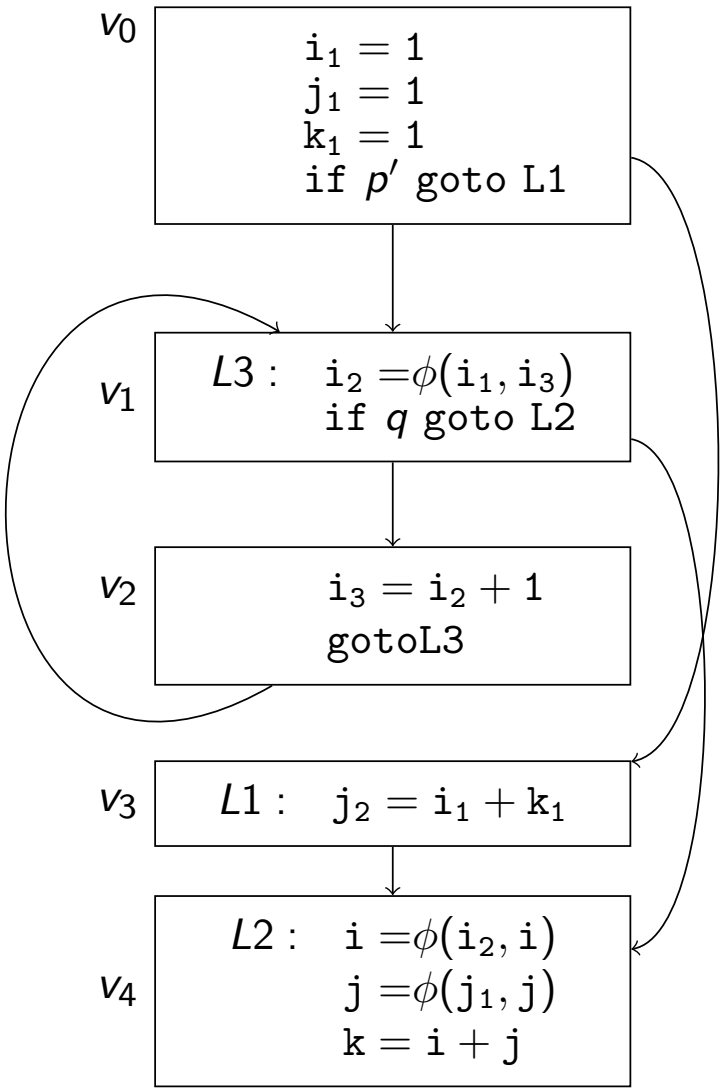
SSA 形式への変換

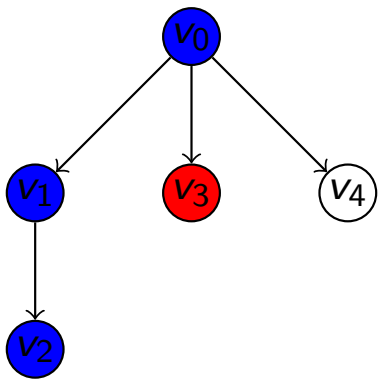




$stack(i) = [1, 2, 3]$ $c(i) = 4$
$stack(j) = [1]$ $c(j) = 2$
$stack(k) = [1]$ $c(k) = 2$

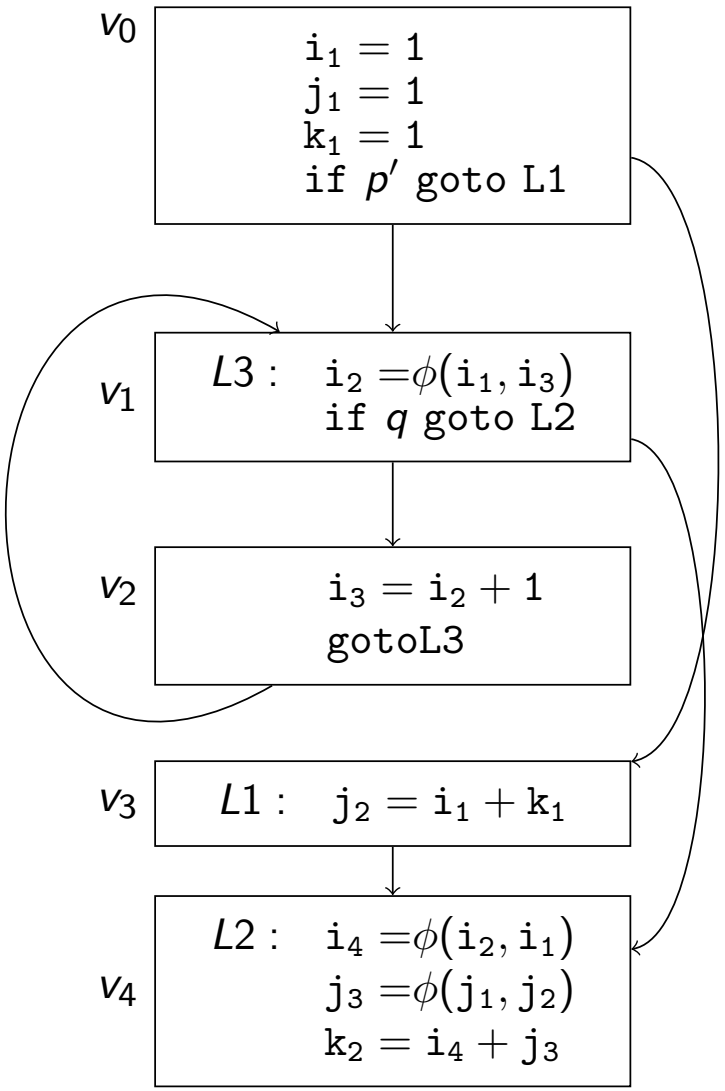
SSA 形式への変換

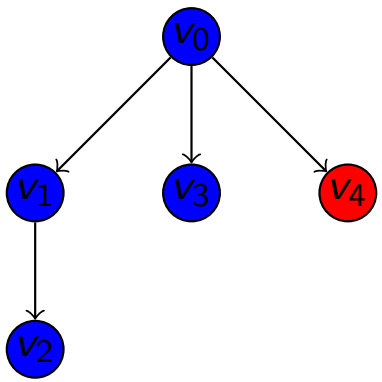




$stack(i) = [1, 2, 3]$ $c(i) = 4$
$stack(j) = [1, 2]$ $c(j) = 3$
$stack(k) = [1]$ $c(k) = 2$

SSA 形式への変換





$stack(i) = [1, 2, 3]$ $c(i) = 4$
$stack(j) = [1, 2]$ $c(j) = 3$
$stack(k) = [1, 2]$ $c(k) = 3$

SSA 形式に対する最適化

SSA 形式：データフロー解析ずみのプログラム

構文的にフローが表現されている ⇒ 最適化を効果的に適用できる：更新のチェックが不要
例：DAG の構成が容易

- 定数伝播
- コピー伝播
- 共通部分式の削除
- ループ不変コードの移動
- 不用コードの削除

SSA 最適化例 [佐々:06]

```
a=1;                    a1=1;
b=2;                    b1=2;
x=3;                    x1=3;
y=a+b;                  y1=a1+b1;
c=x+y;                  c1=x1+y1;
if(a<10)                if(a1<10)
   z=a+b;                  z1=a1+b1;
else                    else
   z=a+b;                  z2=a1+b1;
d=x+z;                  z3=φ(z1,z2);
println(d);      ⇒      d1=x1+z3;
                        println(d1);
```

SSA 形式へ変換

SSA 最適化例 [佐々:06]

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=a1+b1;
else
   z2=a1+b1;
z3=φ(z1,z2);
d1=x1+z3;
println(d1);
```

$\Rightarrow$

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1(=φ(y1,y1));
d1=x1+z3;
println(d1);
```

共通部分式除去

SSA 最適化例 [佐々:06]

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1(=φ(y1,y1));
d1=x1+z3;
println(d1);
```

⇒

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1;
d1=x1+y1;
println(d1);
```

コピー伝播

SSA 最適化例 [佐々:06]

```
a1=1;                          a1=1;
b1=2;                          b1=2;
x1=3;                          x1=3;
y1=a1+b1;                      y1=3;
c1=x1+y1;                      c1=6;
if(a1<10)                      if(a1<10)
   z1=y1;                         z1=y1;
else                           else
   z2=y1;                         z2=y1;
z3=y1;                         z3=y1;
d1=x1+y1;                      d1=c1;
println(d1);          ⇒        println(d1);
```

共通部分式除去、定数伝播

SSA 最適化例 [佐々:06]

```
a1=1;
b1=2;
x1=3;
y1=3;
c1=6;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1;
d1=c1;
println(d1);
```

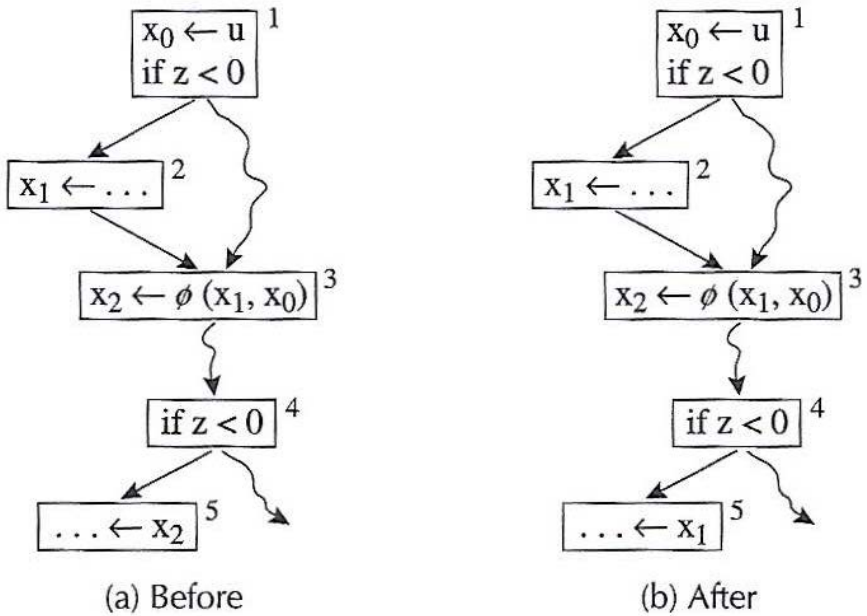
$\Rightarrow$

```
a1=1;
b1=2;
x1=3;
y1=3;
c1=6;
z3=y1;
println(6);
```

コピー伝播、定数伝播、恒等式

最適化の条件

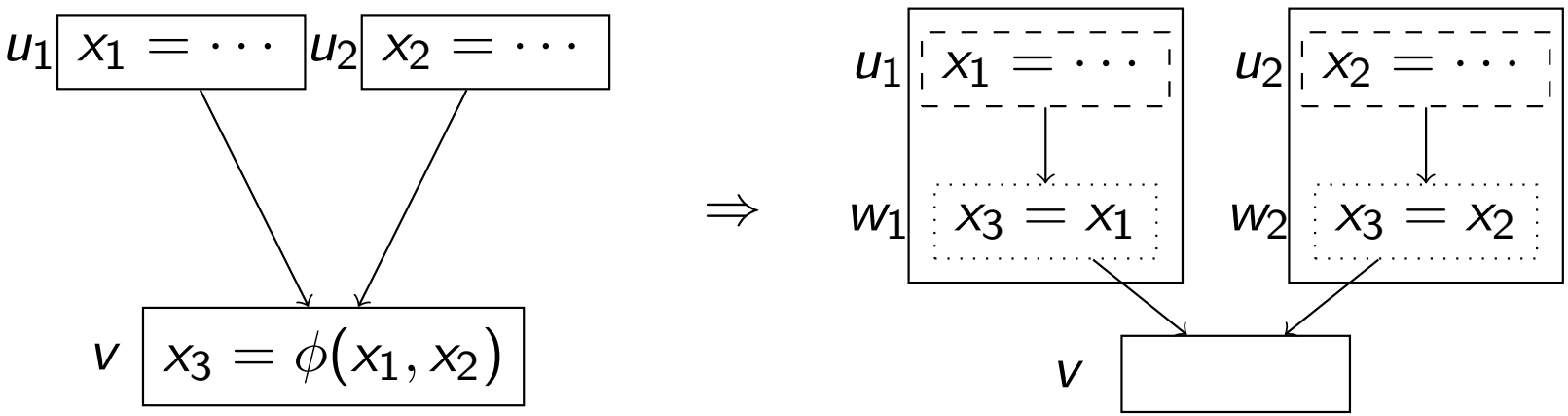
最適化を行うときに支配関係を壊さないようにする。



$z < 0$ の評価は同じ方向にしかブランチしない
$\Rightarrow$ $z < 0$ ならば 3 の ϕ 関数はいつも x_1 を x_2 へ代入
5 において $z < 0$ で、x_1 を参照に用いると支配関係が壊れる

厳密 SSA 形式：**定義が一意的＋支配関係**

SSA 形式からの逆変換：ϕ 関数の消去



ϕ 関数に対してはコード生成ができない。

(先の例はたまたま ϕ 関数まで消えてしまった。)

ノード w_1, w_2 を挿入して、ϕ 関数を消去

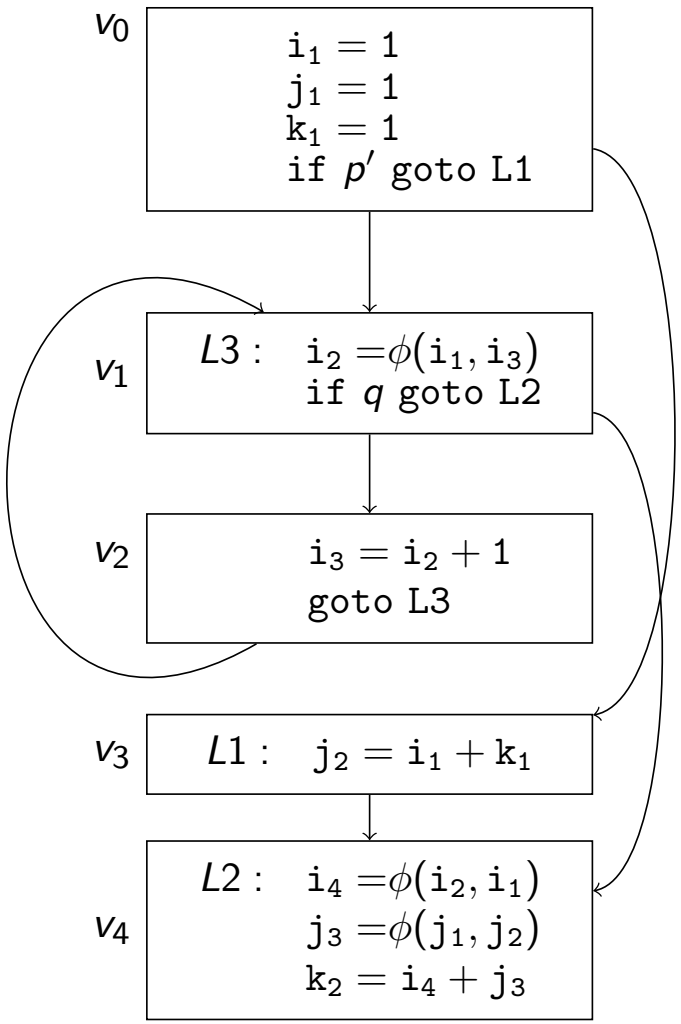
3 番地コード →SSA 形式 → 最適化 → ϕ 関数消去

逆変換アルゴリズム

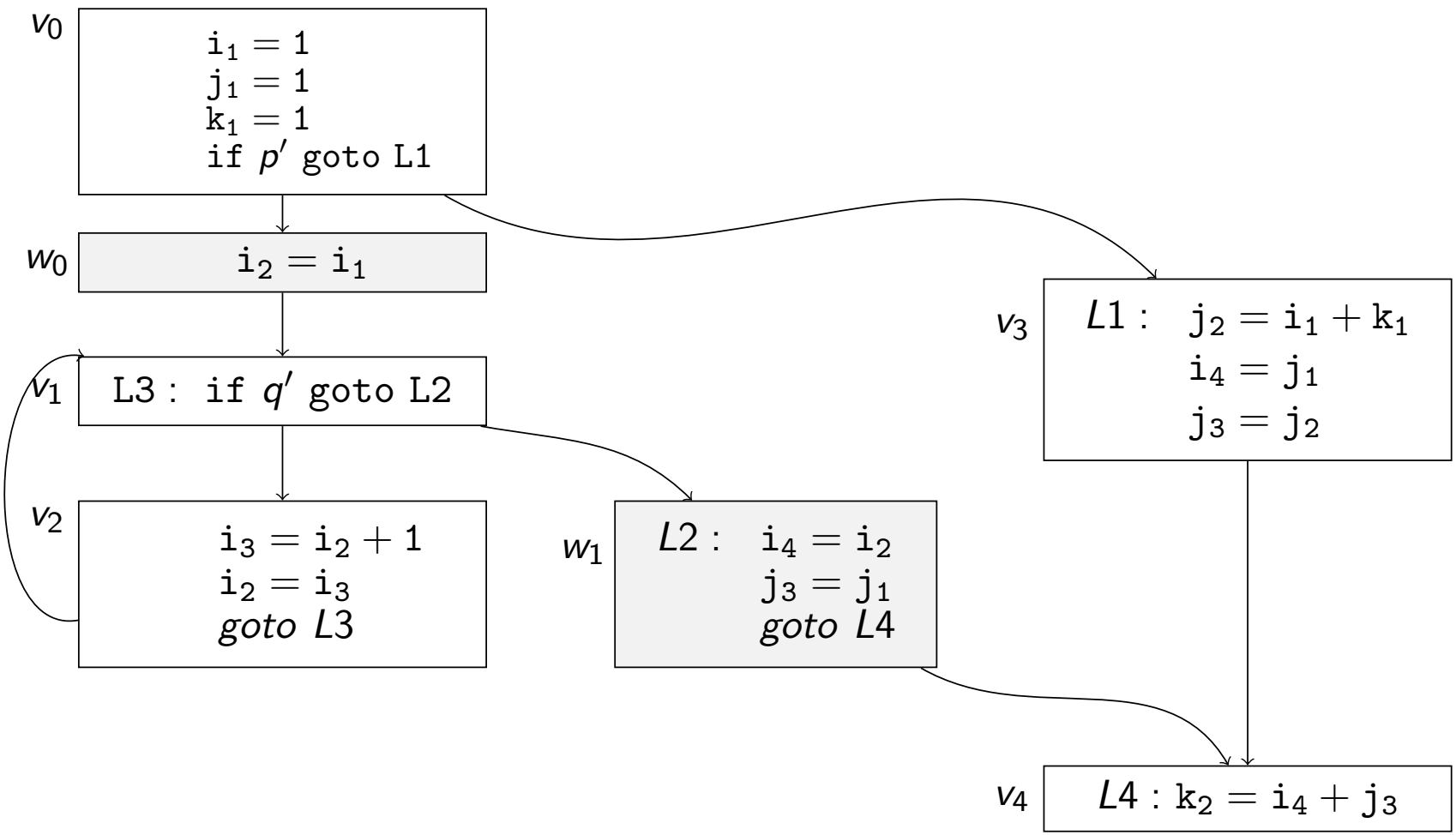
1. フローグラフの変換
   ϕ 関数を持つブロックに対して挿入条件を満たさないまで、代入文を挿入する。

   挿入条件：ϕ 関数を持つブロックが複数の先行頂点を持ち、そのうちの一つが複数の後続頂点を持つ。
   先行ブロックの最後が goto 文の場合は、goto 文の直前に代入文を挿入する。
2. ϕ 関数の消去

逆変換例


v_0
$i_1 = 1$
$j_1 = 1$
$k_1 = 1$
if p' goto L1
v_1
$L3:\ i_2 = \phi(i_1, i_3)$
if q goto L2
v_2
$i_3 = i_2 + 1$
goto L3
v_3
$L1:\ j_2 = i_1 + k_1$
v_4
$L2:\ i_4 = \phi(i_2, i_1)$
$j_3 = \phi(j_1, j_2)$
$k_2 = i_4 + j_3$

逆変換例


v_0
$i_1 = 1$
$j_1 = 1$
$k_1 = 1$
if p' goto L1
w_0
$i_2 = i_1$
v_1
L3 : if q' goto L2
v_2
$i_3 = i_2 + 1$
$i_2 = i_3$
goto $L3$
w_1
$L2$: $i_4 = i_2$
$j_3 = j_1$
goto $L4$
v_3
$L1$: $j_2 = i_1 + k_1$
$i_4 = j_1$
$j_3 = j_2$
v_4
$L4 : k_2 = i_4 + j_3$